

SENA — ADSO 2026

Clase 33

Diseño Técnico del Proyecto

API de Eficiencia de Operarios

Texticode — Sistema de Gestión Textil

Aprendices

Kevin Stevan Wagner
Kevin Andres Castro
Andres Felipe Lancheros
Camilo Andres Tibambre

Instructor

Ing. Edwin Marín
Fecha
Septiembre
2026
Programa
ADSO 2026

1. Arquitectura 3 Capas

Texticode implementa una arquitectura REST en capas. El cliente web Vue.js y el cliente móvil Flutter se comunican con Express mediante HTTP; los endpoints de eficiencia y carga de trabajo usan x-api-key. El acceso de la aplicación a login y Google Calendar emplea JWT Bearer. El frontend no consulta directamente Supabase.

FRONTEND WEB Y MÓVIL
Vue.js 3 · EficienciaOperarios.vue · Axios · VITE_API_KEY Flutter · texticode_mobile · paquete http · ApiConstants.eficienciaHeaders envía x-api-key URL base móvil: http://192.168.1.61:3001/api · configurable con --dart-define=API_BASE_URL
BACKEND
BASE DE DATOS
Supabase PostgreSQL · usuario, rol, orden_produccion, orden_operario, observacion_operario, material, orden_material y tablas adicionales descritas en el ER

FLUJO DE DATOS — CONSULTAR EFICIENCIA DE OPERARIOS

Capa	Acción
Frontend	EficienciaOperarios.vue → axios.get('/api/eficiencia/operarios', { headers: { 'x-api-key': KEY } })
Backend Auth	middleware apiKey.js valida x-api-key contra API_KEY_EFICIENCIA en .env. El cliente móvil toma el header de ApiConstants.eficienciaHeaders.
Backend Business	eficiencia.js calcula prendas/día con ponderación por Dificultad (Alta=3, Media=2, Baja=1), no por Prioridad. Umbrales de rendimiento Alto/Medio/Bajo: pendientes de confirmar en el código vigente.
Persistence	pg.Pool ejecuta consultas a Supabase. El tipo de pooler y puerto efectivos deben coincidir con .env; 6543 es Transaction Pooler y 5432 Session Pooler.
Database	PostgreSQL devuelve registros con garantías ACID
Response	200 OK + { ok: true, total, filtros_aplicados, data: [...] }
Flujo móvil	texticode_mobile → EficienciaRepository → paquete http → Express (x-api-key) → Supabase → respuesta JSON

2. Contrato Técnico API

Este documento describe los contratos de eficiencia, el módulo móvil Flutter y el módulo de carga de trabajo. Las evidencias incluidas corresponden a las capturas presentes en el documento fuente; las pruebas nuevas se señalan como pendientes de captura.

2.1 GET /api/eficiencia/operarios

GET /api/eficiencia/operarios

Lista todos los operarios activos con métricas de rendimiento calculadas en tiempo real.

REQUEST

GET /api/eficiencia/operarios HTTP/1.1

Host: localhost:3001

x-api-key: texticode-2026

// Query params opcionales

?rendimiento=Alto|Medio|Bajo

?estado=Completada|En Proceso|Pausado

?periodo=YYYY-MM (según Fecha_Limite)

?limite=5

Host: localhost:3001

x-api-key: texticode-2026

// Query params opcionales

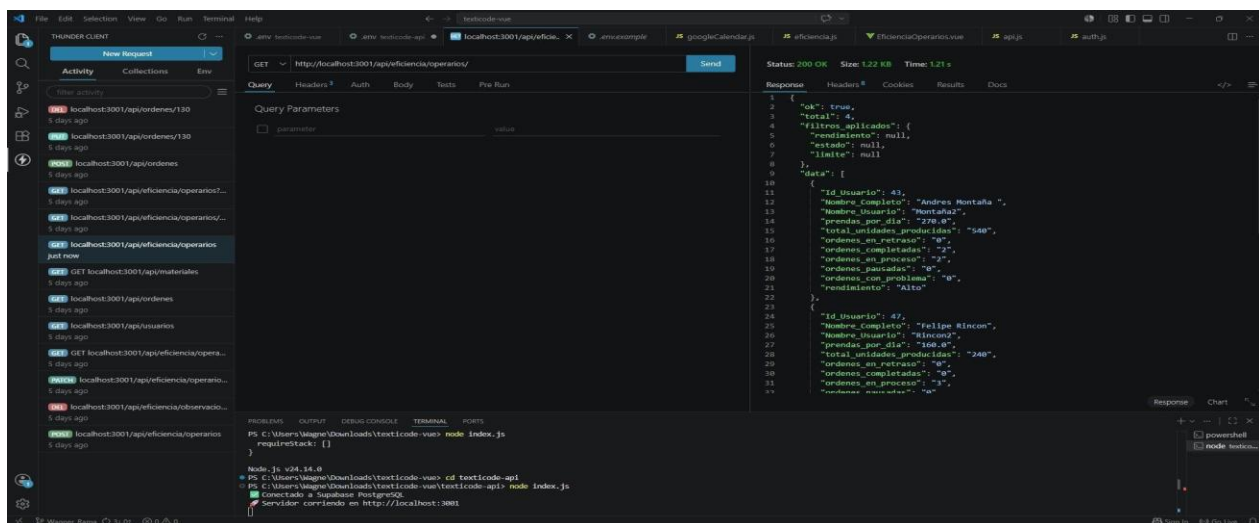
?rendimiento=Alto|Medio|Bajo

?estado=Completada|En Proceso|Pausado

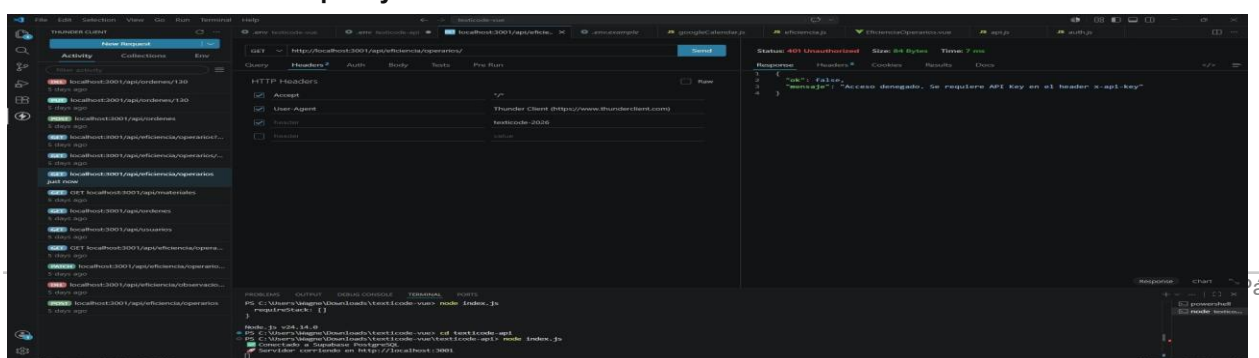
?limite=5

EVIDENCIA THUNDER CLIENT

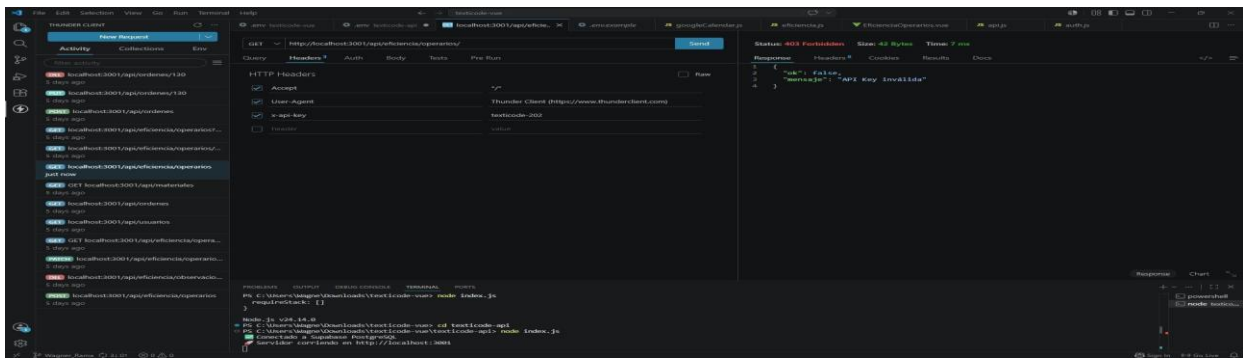
Prueba 1 — API Key correcta → 200 OK



Prueba 2 — Sin header x-api-key → 401 Unauthorized



Prueba 3 — API Key incorrecta → 403 Forbidden



2.2 GET /api/eficiencia/operarios/:id

GET /api/eficiencia/operarios/:id

Devuelve perfil completo de un operario con órdenes, métricas y observaciones.

REQUEST

```
GET /api/eficiencia/operarios/52 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

// :id → Id_Usuario del operario (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Operario existente (id: 52 — Stevan Wagner) → 200 OK

The screenshot shows the Thunder Client interface. The left sidebar lists several requests. The main panel displays the details of a GET request to `http://localhost:3001/api/eficiencia/operarios/52`. The status is 200 OK. The response body is a JSON object with the following structure:

```
{
  "ok": true,
  "data": {
    "id_usuario": 52,
    "nombre_completo": "Stevan Wagner",
    "nombre_usuario": "wagner2",
    "correo": "stevanwagner@gmail.com",
    "telefono": "+57 322 4521303",
    "promedio_por_dia": "113.8",
    "total_unidades_producidas": "113",
    "ordenes_en_proceso": "0",
    "ordenes_completadas": "0",
    "ordenes_en_proceso": "0",
    "ordenes_pendientes": "0",
    "ordenes_con_problemas": "0",
    "rendimiento": "Alto",
    "ordenes_detalle": [
      {
        "id_orden": 143,
        "producto": "Bases",
        "estado": "En Proceso",
        "prioridad": "Alta",
        "dificultad": "Medio",
        "unidades_realizadas": 98,
        "unidades": 100,
        "fecha_limite": "2026-05-30T05:00:00.000Z",
        "verificada": false,
        "tiene_problemas": false,
        "observaciones": []
      }
    ]
  }
}
```

Prueba 2 — Operario inexistente (id: 29) → 404 Not Found

The screenshot shows the Thunder Client interface. The left sidebar lists several requests. The main panel displays the details of a GET request to `http://localhost:3001/api/eficiencia/operarios/29`. The status is 404 Not Found. The response body is a JSON object with the following structure:

```
{
  "ok": false,
  "mensaje": "No se encontró operario con id 29"
}
```

2.3 POST /api/eficiencia/observaciones

POST /api/eficiencia/observaciones

Registra una nueva observación sobre el rendimiento de un operario en una orden específica.

REQUEST

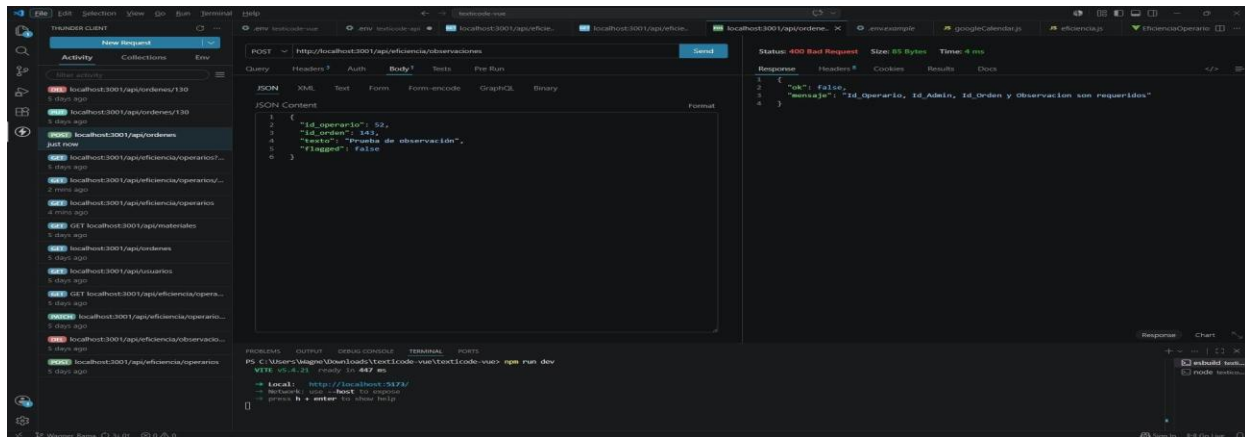
```
POST /api/eficiencia/observaciones HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026
Content-Type: application/json
```

```
{
  "Id_Operario": 52,
  "Id_Admin": <Id_Usuario del administrador autenticado>,
  "Id_Orden": 143,
  "Observacion": "Prueba de observación"
}

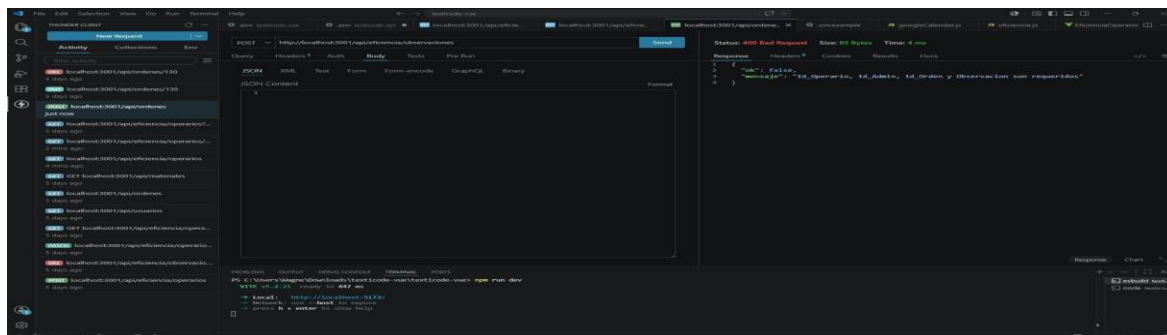
// Todos los campos son requeridos. Observacion máx. 500 caracteres.
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Body completo y correcto → 201 Created



Prueba 2 — Body vacío (campos faltantes) → 400 Bad Request



2.4 DELETE /api/eficiencia/observaciones/:id

DELETE /api/eficiencia/observaciones/:id

Elimina una observación de eficiencia por su Id_Observacion.

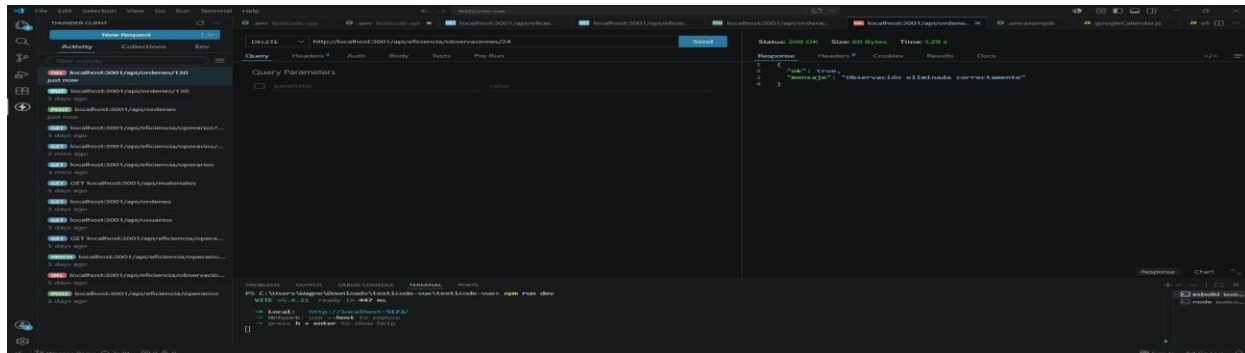
REQUEST

```
DELETE /api/eficiencia/observaciones/24 HTTP/1.1
Host: localhost:3001
x-api-key: texticode-2026

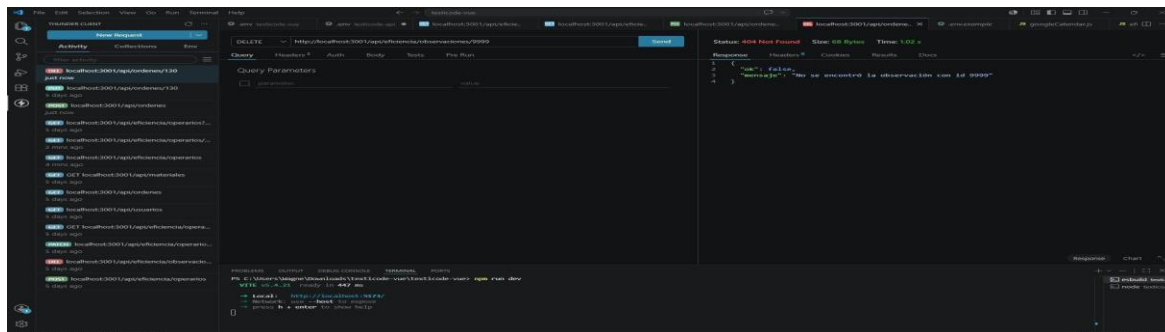
// :id → Id_Observacion (entero positivo)
```

EVIDENCIA THUNDER CLIENT

Prueba 1 — Observación existente (id: 24) → 200 OK



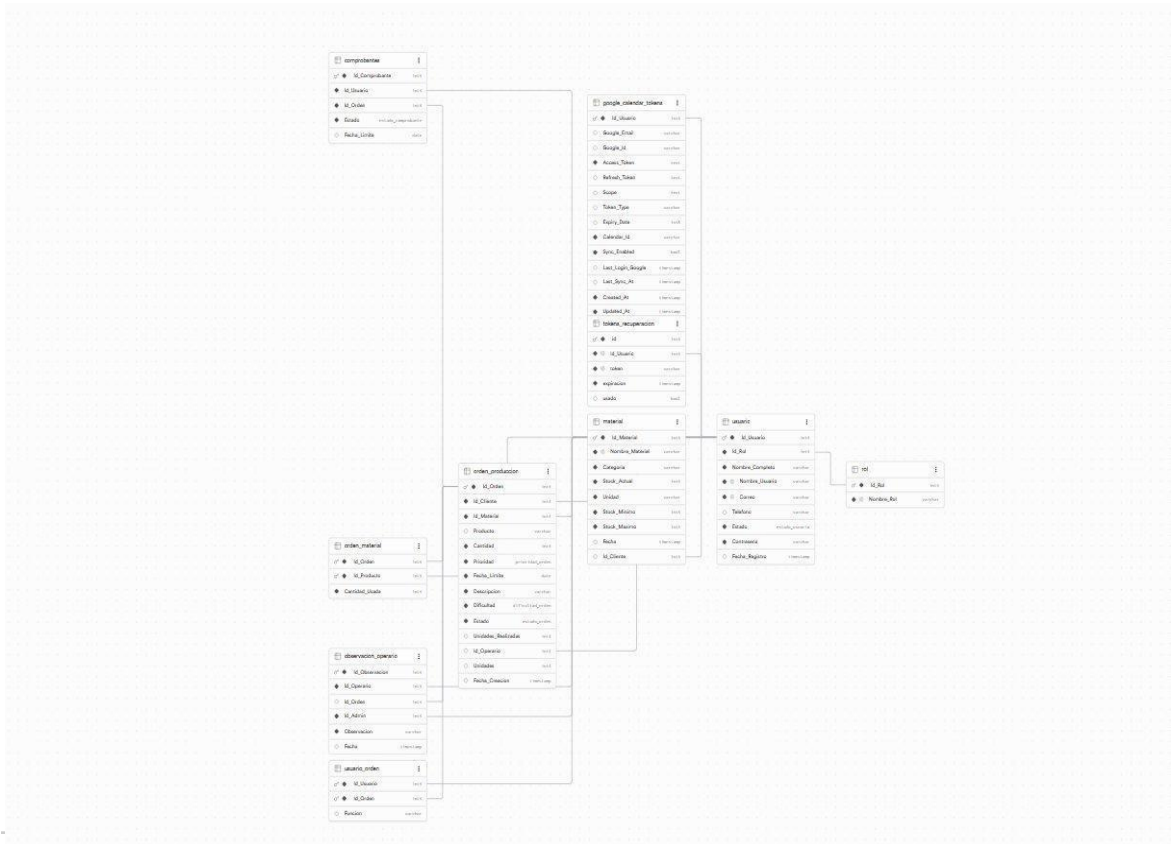
Prueba 2 — Observación inexistente (id: 9999) → 404 Not Found



3. Diagrama Entidad-Relación

El módulo de eficiencia se apoya en las tablas descritas a continuación y en las asignaciones por fase de orden_operario. La sección ER distingue las tablas centrales del módulo de las tablas adicionales usadas por la aplicación.

Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Operario (FK) · Id_Cliente (FK) · Id_Material (FK) · Producto · Cantidad · Prioridad · Dificultad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion
observacion_operario	Id_Observacion (PK) · Id_Operario (FK) · Id_Orden (FK) · Id_Admin (FK) · Observacion · Fecha
material	Id_Material (PK) · Nombre_Material · Categoria · Stock_Actual · Stock_Minimo · Stock_Maximo · Unidad · Fecha · Id_Cliente (FK)
orden_material	Id_Orden (FK) · Id_Producto (FK → material.Id_Material) · Cantidad_Usada
orden_operario	Id_Orden_Operario (PK) · Id_Orden (FK) · Id_Operario (FK) · Numero_Fase · Descripcion_Fase · Cantidad_Realizada · Estado_Fase · Nota_Operario · Fecha_Completada
comprobante	Id_Comprobante (PK) · Id_Usuario (FK) · Id_Orden (FK) · Estado · Fecha_Limite
material_movimiento	Bitácora de movimientos de inventario; campos físicos por confirmar en el esquema vigente.
tokens Google Calendar	Tabla de tokens OAuth de Calendar; nombre y columnas por confirmar en el esquema vigente.



RELACIONES CLAVE

Origen	Relación	Destino	Descripción
usuario	N:M vía orden_operario	orden_produccion	La asignación por fases se registra en orden_operario. orden_produccion.Id_Operario se conserva como referencia al primer operario.
usuario	1 → N	observacion_operario	Un operario puede acumular múltiples observaciones
orden_produccion	1 → N	observacion_operario	Una orden puede tener múltiples observaciones
rol	1 → N	usuario	Un rol agrupa múltiples usuarios
material	1 → N	orden_produccion	Un material puede usarse en múltiples órdenes
orden_produccion	1 → N	orden_material	Una orden registra el uso de materiales
usuario	1 → N	orden_operario	Cada operario puede cubrir varias fases; una orden puede asignarse a varios operarios por fase.
orden_produccion	1 → N	orden_operario	Cada orden puede tener varias fases/asignaciones.
material	1 → N	orden_material	orden_material.Id_Producto referencia material.Id_Material.

4. Validaciones y Manejo de Errores

Los endpoints de eficiencia y carga de trabajo protegidos comparten el middleware `apiKey.js`. La aplicación también usa JWT Bearer para login y Calendar; otros endpoints como `/usuarios` y `/ordenes` no exigen API Key ni JWT en las llamadas de la app descritas aquí.

Escenario	Validación en código	HTTP
API Key ausente	<code>!req.headers['x-api-key']</code>	401 Unauthorized
API Key incorrecta	<code>key !== process.env.API_KEY_EFICIENCIA</code>	403 Forbidden
ID no numérico	<code>isNaN(id)</code>	400 Bad Request
Operario no encontrado	<code>rows.length === 0</code>	404 Not Found
Observación no encontrada	<code>obs.length === 0</code>	404 Not Found
Campos requeridos faltantes	<code>!Id_Operario !Id_Admin !Id_Orden !Observacion</code>	400 Bad Request
Observacion > 500 chars	<code>Observacion.length > 500</code>	400 Bad Request
Orden no pertenece al operario	<code>SELECT orden WHERE Id_Operario = ? → rowCount 0</code>	400 Bad Request
Error interno de BD	<code>catch(err) → log interno</code>	500 Server Error
Periodo inválido en historial	Periodo distinto de semana mes trimestre	400 Bad Request (confirmar respuesta exacta en backend)
Llave foránea inválida en observación	Id_Admin o Id_Orden inexistente; se espera violación FK	Código HTTP por confirmar en backend; documentar mensaje en campo mensaje

MIDDLEWARE APIKEY.JS

```
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key)
    return res.status(401).json({ ok: false,
      mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key inválida' })
  next()
}
export default verificarApiKey
```

Ampliación de contratos y configuración

Esta ampliación completa los contratos consumidos por los clientes web y móvil. Los estados calculados se distinguen de los valores persistidos. Las capturas de Thunder Client que no existían en el documento fuente se dejan como espacios de evidencia pendientes.

GET /api/eficiencia/operarios

Query params opcionales: `rendimiento=Alto|Medio|Bajo`; `estado=Completada|En Proceso|Pausado`; `periodo=YYYY-MM`; `limite=entero`. `periodo` filtra las órdenes por `Fecha_Limite` dentro del mes indicado y debe reflejarse en `filtros_aplicados`. `Retrasada` es una condición calculada por fecha de vencimiento, no un valor persistido de Estado.

La respuesta expone `total`, `filtros_aplicados` y `data[]`. Cada operario incluye `prendas_por_dia`, `total_unidades_producidas`, `ordenes_en_retraso`, `ordenes_completadas`, `ordenes_en_proceso`,

ordenes_con_problema, rendimiento y ordenes_detalle. No se documenta ordenes_pausadas porque el backend no lo devuelve.

ordenes_detalle[]: Id_Orden, Producto, Estado, Prioridad, Dificultad, Unidades_Realizadas, Unidades, Fecha_Limite, vencida, tiene_problema y observaciones[]. Cada observación contiene Id_Observacion, Observacion, Fecha y Admin.

GET /api/eficiencia/operarios/:id

Devuelve el perfil del operario con las métricas y ordenes_detalle descritas arriba. El campo vencida se calcula usando Fecha_Limite; tiene_problema refleja la presencia de una observación asociada según la lógica del backend.

GET /api/eficiencia/operarios/:id/historial

Consulta: GET /api/eficiencia/operarios/:id/historial?periodo=semana|mes|trimestre. Respuesta: { ok, data: { periodo, dias, actual, anterior, tendencia, diferencia_prendas } }. actual y anterior contienen prendas_por_dia, total_unidades, completadas, en_curso, retrasos, con_problema y rendimiento. tendencia admite subiendo, bajando o estable.

Pruebas previstas en Thunder Client: periodo válido → 200; sin x-api-key → 401; clave incorrecta → 403; operario inexistente → 404; periodo inválido → 400 según matriz pendiente de contrastar con el código. Insertar aquí captura de cada respuesta al ejecutar las pruebas.

POST /api/eficiencia/observaciones

Id_Admin debe ser el identificador real del administrador autenticado obtenido desde AuthRepository; no se debe enviar un valor fijo. El backend retorna los errores en el campo mensaje, que es el nombre leído por la app. Los campos obligatorios son Id_Operario, Id_Admin, Id_Orden y Observacion (máximo 500 caracteres). Si Id_Admin o Id_Orden no existe, se produce una violación de llave foránea; el código HTTP y el mensaje exactos requieren confirmación en el backend vigente.

Cálculo de eficiencia

La ponderación Alta=3, Media=2 y Baja=1 corresponde al campo Dificultad. La Prioridad es un campo separado. No se incluyen umbrales para clasificar rendimiento Alto, Medio o Bajo porque no se aportó el código que los determina; quedan pendientes de verificar. También debe revisarse si prendas/día se atribuye a orden_produccion.Id_Operario o a las fases de orden_operario.

Módulo de carga de trabajo

Los cinco endpoints siguientes usan el header x-api-key. El filtro estado acepta sobrecargado, normal o disponible.

Endpoint

GET /api/carga-trabajo?estado=

Contrato

Devuelve resumen{total_operarios, sobrecargados, normales, disponibles} y data[]. Cada operario incluye ordenes_activas, fases_activas, ordenes_vencidas, ordenes_alta_prioridad y estado_carga.

GET /api/carga-trabajo/operarios/:id

Incluye data.ordenes_activas_detalle[]. Campos internos del detalle: completar desde el esquema del backend y agregar evidencia.

GET /api/carga-trabajo/sugerencias

Devuelve sugerencias[]. Estructura detallada pendiente de confirmar en la respuesta real.

POST /api/carga-trabajo/reasignar

Body acepta Id_Orden_Operario o Id_Orden, junto con Id_Operario_Destino.

POST /api/carga-trabajo/aplicar-sugerencias

Devuelve { aplicados }.

EVIDENCIA PENDIENTE: insertar capturas Thunder Client de solicitud y respuesta de cada endpoint, incluyendo estados de carga y validaciones.

Modelo de datos y asignación por fases

La lista inicial contiene seis tablas: usuario, rol, orden_produccion, observacion_operario, material y orden_material. Se agregan las tablas usadas por la aplicación: orden_operario, comprobante, material_movimiento y almacenamiento de tokens de Google Calendar. Los nombres y campos no proporcionados se deben cotejar con el esquema vigente antes de cerrar el diagrama.

orden_operario registra Id_Orden_Operario, Id_Orden, Id_Operario, Numero_Fase, Descripcion_Fase, Cantidad_Realizada, Estado_Fase, Nota_Operario y Fecha_Completada. La relación orden-producción/operario es N:M a través de esta tabla; orden_produccion.Id_Operario representa actualmente al primer operario. Debe validarse si el cálculo de eficiencia considera las fases.

orden_produccion admite los estados persistidos En Proceso, Completada y Pausado. Retrasada es un estado derivado de la fecha límite. En orden_material, Id_Producto referencia material.Id_Material. comprobante incluye Id_Comprobante, Id_Usuario, Id_Orden, Estado y Fecha_Limite.

Seguridad, errores y configuración

El cliente móvil envía x-api-key desde ApiConstants.eficienciaHeaders. En api_constants.dart la clave texticode-2026 está fija y debe coincidir con API_KEY_EFICIENCIA del servidor. Es una limitación conocida: una clave incorporada en el APK puede recuperarse al descompilarlo; no debe considerarse un secreto de cliente confiable.

La URL base móvil predeterminada es http://192.168.1.61:3001/api y puede configurarse con --dart-define=API_BASE_URL. En .vscode/launch.json, GOOGLE_WEB_CLIENT_ID es un placeholder; api_constants.dart tiene un valor por defecto. Deben alinearse antes de usar Calendar en cada entorno.

API Key protege los endpoints de eficiencia y carga de trabajo. Login y Calendar usan JWT Bearer. Las llamadas de la app a /usuarios, /ordenes y otros endpoints indicados no exigen API Key ni JWT; confirmar el alcance exacto con las rutas desplegadas.

Documentación del repositorio

README-Google-API.md conserva la URL antigua http://192.168.0.6:3001/api, mientras el cliente usa http://192.168.1.61:3001/api. Debe actualizarse al valor del entorno o referir la variable configurable API_BASE_URL.

README.md y README-Google-API.md describen de forma distinta las respuestas de DELETE /calendar/connect y POST /calendar/sync/:idOrden. La respuesta canónica debe extraerse de los handlers actuales; como el código no se adjuntó, no se inventa una respuesta única. Conciliar ambos README tras revisar el backend.

Compatibilidad del modelo Flutter

EficienciaOperario.ordenesPausadas no corresponde a una propiedad enviada por el backend. Retirar el campo del modelo o mantenerlo explícitamente como no soportado y no usarlo para presentar datos.